UE23CS441A – Capstone Project Phase – 3

Project Progress Review # 1

Project Title : A Unified Framework for Deep Multimodal Audio-Visual Steganography
Project ID : PW26_SBN_07
Project Guide : Dr. Surabhi Narayan
Project Team :

- Karthik Prabhu
- Harish Srinivasan
- Kappagantula Sai Manogna
- K S Arulmozhi Varma

Abstract and Scope

- **PROBLEM STATEMENT** - Design a deeplearning-based **multimodal** steganography framework that can **jointly embed** and **extract** a high-capacity secret payload (especially a full video, including imagery and audio) within a host video by hiding information **across both the video's frames and its audio track**, while maintaining imperceptibility in each modality and robustness against common distortions (e.g. compression).

- **SCOPE** :
  - Implement Audio-INN + Video-INN as modular components
  - Per-chunk encode/decode loop (no drift, consistent k indexing)
  - Time-synchronous chunking
  - Process the clip in fixed sized chunks
  - Training + evaluation:
    - Stego quality (audio/video similarity vs cover)
    - Secret recovery quality (audio/video similarity vs secret)
    - Runtime feasibility on a single GPU setup

Literature Survey

Hide and Speak (Speech Steganography)

- Proposes a deep speech steganography pipeline (Encoder + Carrier Decoder + Message Decoder) to hide and recover secret speech from carrier audio.
- Key contribution: differentiable STFT–iSTFT inside training to address spectrogram–waveform mismatch.
- Supports multiple secret messages per carrier, with human ABX tests and WER/CER for intelligibility.
- Limitations:
  - No chunk-wise encoding or A/V synchronization framework.
  - Not cleanly reversible (STFT phase reconstruction issues).
  - Scalability trade-off: quality degrades as number of hidden messages increases.
  - Sensitive to quantization and compression, reducing deployment robustness.

LF-VSN (Video Steganography via INNs)

- Uses a single Invertible Neural Network (INN) for both hiding and revealing, enabling reversible, large-capacity video steganography.
- Improves imperceptibility using short temporal windows + frequency-domain fusion.
- Introduces Redundancy Prediction Module (RPM) for stable recovery.
- **Limitations**:
  - Video-only (no cross-modal A/V sync).
  - Real-world compression can threaten perfect reversibility.

Key Takeaway for Our Work

- Borrow: INN-based reversible design, short-window temporal context, redundancy prediction.
- Gap Identified: No prior work offers a joint audio–video steganography system with strict time-synchronized chunking.
- Our Novelty: A unified multimodal framework ensuring synchronized, chunk-wise audio–video hiding with scalable capacity and stable recovery

Summary of Work Done in Capstone Project Phase - 2

- Explored CNN-based video steganography as an alternative approach.
- CNN processes video frame-by-frame and provides only approximate recovery.
- Explored STFT/iSTFT-based audio steganography for deep-learning-based processing.
- iSTFT reconstruction can introduce distortion and information loss in the recovered audio.
- INN-based approach provides mathematically reversible transformations with better reconstruction quality.
- Hence, INN was selected for its reversibility, quality, and reliability.

Overall Architecture

- VIDEO INN
- AUDIO INN
- Temporal Synchronisation

Video-in-Video Steganography INN

Goal: Hide one secret video inside a cover video using an Invertible Neural Network (INN) so decoding can near-exactly recover both.
Windowing (temporal context): Use a sliding window of length L around time t. The encoder processes the cover+secret window but outputs only the center stego frame **stego[t]**.
Feature Fusion (to reduce concat artifacts): A swappable TransformFusion module:

- **Fixed option**: Haar wavelet + learnable 1×1 conv mixing
- **Learnable option**: lifting / pseudo-wavelet via strided convolutions

Invertible Core (two-stream coupling): Operates on Cover features C and Secret features S with reversible coupling blocks:

- **Additive coupling**: C_{k+1} = C_k + g_s(S_k)
  Affine coupling: S_{k+1} = S_k * exp(clamp(g_scale(C_{k+1}))) + g_shift(C_{k+1})
- Uses lightweight subnets (e.g., depthwise separable / small 3D convs) + invertible 1×1 permutations between blocks.

Redundancy Estimator (key trick for single-frame output):

- In real invertible hiding, the stego frame may not be enough by itself to perfectly reverse everything — so the method uses an extra "helper" latent called Z (redundancy).
- Instead of sampling , predict missing redundancy $\hat{z}$ from stego context using a small residual CNN .
- Feed $\hat{z}$ into the reverse pass to reconstruct the secret.

Decoder "broadcast trick": Since decoder expects a window-shaped input, repeat stego[t] into a pseudo-group, can be modified so each copy is distinguishable.
Training objective (single-secret):

- **Forward loss**: keep stego close to cover (L1 + (1 − MS-SSIM))
- **Backward loss**: reconstruct secret (and optionally cover) (L1 terms)

What you get:

- Encoder: cover + secret → stego (visually similar to cover)
- Decoder: stego → recovered secret (+ recovered cover) via INN reverse + predicted $\hat{z}$

➢ Audio-in-Audio Steganography INN

1) Audio Preprocessing

- Cover and secret audio resampled to **16 kHz**
- Segmented into **1-second chunks** (16,000 samples)
- Scrambling within a secret chunk (simple encryption)
- Normalized and paired as network input

2) Invertible Neural Network (INN)

- Input: *(cover chunk x1, secret chunk x2)*
- Processed through a sequence of **invertible blocks**
- Each block uses:
  - **Coupling layers** (split, scale(s) , shift(t))
  - **Affine transformations**
  - **Channel shuffling** for full mixing

$$y_2 = x_2 \odot \exp(s) + t$$

$$x_2 = (y_2 - t) \odot \exp(-s)$$

3) Output Generation

- **Stego audio (y1)** : perceptually similar to cover, contains hidden secret
- **Auxiliary latent variable (y2)** : regularized to follow Gaussian distribution $\mathcal{N}$**(0, I) ;** Stores excess information for invertibility

4) Time–Frequency Loss (Applied during forward pass) : The model calculates the difference between the original cover and the generated stego audio in both the time domain (raw samples) and the frequency domain (using STFT). This ensures that the embedding process doesn't just minimize mathematical error, but also minimizes audible artifacts that the human ear might detect.

Temporal Synchronization Module

- Audio and video steganography are performed by *independent frameworks*, which can cause **audio–video drift** if temporal correspondence is not explicitly enforced
- We chunk both audio and video into fixed 1-second segments. Audio chunks consist of exactly 16,000 samples at 16 kHz, while video frames are grouped by timestamp using floor(time). All embedding and extraction is performed independently per chunk, ensuring strict temporal alignment.

Chunk Index Assignment

- Each 1-second interval is assigned a **global chunk index *i***, such that : `VideoChunk_i ↔ AudioChunk_i`
- This index represents the *same real-time interval* in both streams ensuring **one-to-one temporal correspondence**

For each index *i*:

- SecretVideoChunk_i is embedded into CoverVideoChunk_i
- SecretAudioChunk_i is embedded into CoverAudioChunk_i

Then, Concatenate all stego frames and Concatenate all stego audio chunks in time order

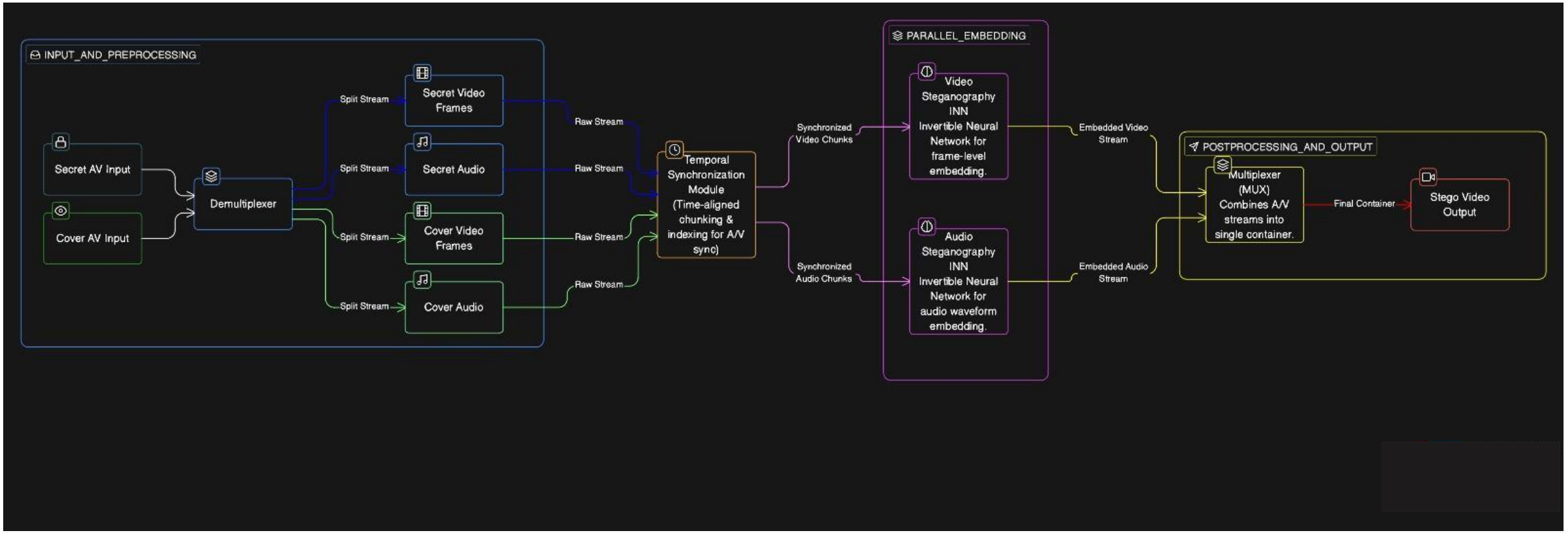
Decoding & Reconstruction

- During extraction, the same chunk indexing is applied, allowing recovered audio and video chunks with the same index to be recombined without drift even incase of variable frame rates.

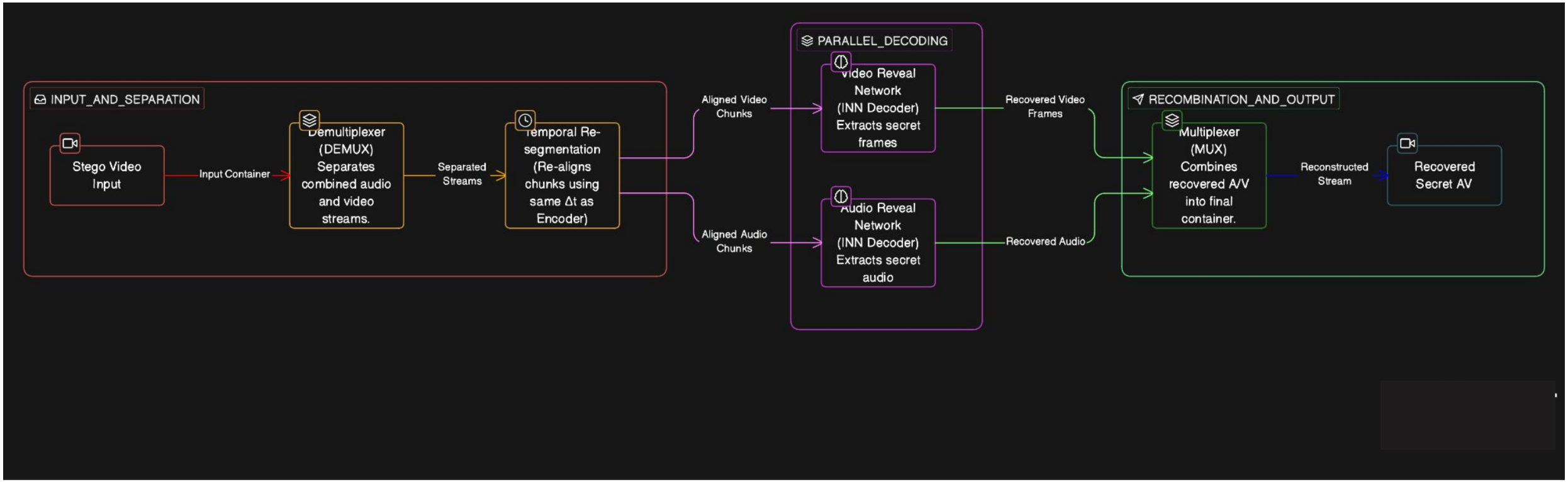
Why is this synchronization required?

- Say, first 200 ms of stego video removed, can I still decode aligned chunks? YES because with chunk indexing implemented decoding starts from next complete 1-second chunk
- Stego video is re-encoded from **30 FPS → 29.97 FPS** can I still keep alignment? YES since chunking is performed using time-based segmentation.

ENCODER ARCHITECTURE



DECODER ARCHITECTURE



List of tasks/Modules

Video Dataset Preparation
Use Vimeo90K video sequences and form cover-secret video pairs.

Video Pre-processing
Load frames, normalize pixels, crop 144 x 144 regions, and apply augmentation.

Temporal Windowing
Select three consecutive frames to provide local motion information.

Haar Wavelet Transformation
Convert RGB frames into reversible low- and high-frequency feature components.

Video-INN Encoder
Embed secret video features into cover features using invertible coupling blocks.

Latent Prediction and Video Decoder
Predict latent features and run the invertible network in reverse to recover the secret.

Quantization and Stego Generation
Simulate 8-bit storage and generate the final stego frame/video.

Training and Performance Evaluation
Train using reconstruction losses and evaluate using PSNR and SSIM.

Audio-INN and AV Synchronization
Implement the proposed audio pipeline and synchronize audio-video segments.
Audio training and complete synchronization are currently pending.

SDKs / APIs / Models / Tools / Technologies

Python
Main programming language.

PyTorch
Neural-network development and model training.

CUDA and cuDNN
GPU acceleration for training and inference.

OpenCV
Reading, processing, and saving video frames.

NumPy
Numerical processing and array manipulation.

PyYAML
Reading training and model configuration files.

TensorBoard and Matplotlib
Training monitoring, visualization, and result plotting.

Gradio
User interface for the video steganography demonstration.

Individual contribution

Team Member	Tasks / Modules Assigned	Development	Time Spent
Karthik Prabhu	Vimeo90K dataset preparation, cover-secret pairing, temporal windowing, and training data pipeline	~358 LOC	~3 weeks
Harish Srinivasan	Haar DWT/IWT implementation, image preprocessing, augmentation, and quantization module	~317 LOC	~2 weeks
Kappagantula Sai Manogna	Video-INN encoder, affine coupling blocks, dense convolutional subnetworks, and training-loss integration	~373 LOC	~2 weeks
K S Arulmozhi Varma	Latent prediction module, reverse Video-INN decoder, testing pipeline, PSNR/SSIM evaluation, and demo application	~391 LOC	~3 weeks

Shared Contributions

- LF-VSN literature study and architecture selection.
- Training configuration and experiment setup.
- Debugging and model integration.
- Checkpoint testing and result verification.
- Visualization, result interpretation, report writing, and presentation preparation.

Demonstration and Testing of the Modules Completed

Metrics :

- PSNR (Peak Signal-to-Noise Ratio)
  - Measures pixel-level difference between two frames.
- SSIM (Structural Similarity Index)
  - Measures the difference between original and recovered audio.

Latest Video Results

Evaluation	PSNR	SSIM
Recovered cover vs. original cover	51.66 dB	0.9953
Recovered secret vs. original secret	49.19 dB	0.9965
Stego vs. original cover	42.74 dB	0.9622

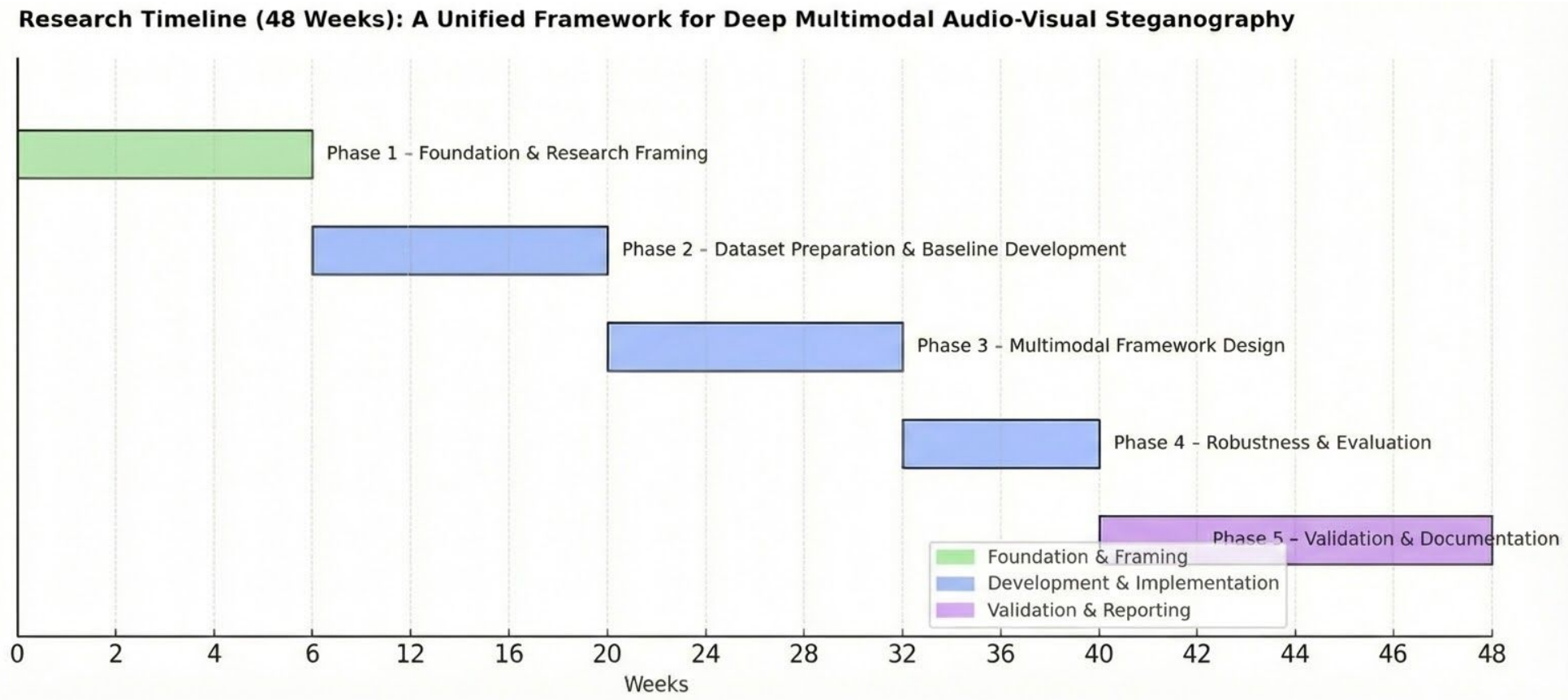
```
### Test run summary

- Latest test summary file: `code/
results/train_LF-VSN_1video_proto/
metrics_test_summary.json`
- Checkpoint used in that test: `.../
models/19000_G.pth`
- Test sample count: `200`
- Test metrics:
  - PSNR Cover: `22.6197`
  - PSNR Secret: `22.5737`
  - PSNR Stego: `22.5418`
  - SSIM Cover: `0.7103`
  - SSIM Secret: `0.7069`
  - SSIM Stego: `0.6857`
```

References

- Mou, C., Xu, Y., Song, J., Zhao, C., Ghanem, B., & Zhang, J. (2023, April 24). Large-capacity and flexible video steganography via invertible neural network. arXiv.org. https://arxiv.org/abs/2304.12300
- Kreuk, F., Adi, Y., Raj, B., Singh, R., & Keshet, J. (2019, February 7). Hide and Speak: towards deep neural networks for speech steganography. arXiv.org. https://arxiv.org/abs/1902.03083
- Islam, Saiful & Nigam, Aditya & Mishra, Aayush & Kumar, Suraj. (2019). VStegNET: Video Steganography Network using Spatio-Temporal features and Micro-Bottleneck.
- WAVINWAV: Time-Domain speech hiding via invertible neural network. (n.d.). https://arxiv.org/html/2510.02915v1

Project Progress Plan for Phase 3 and Phase 4



THANK YOU